

Misleading Identifiers Change What Code Traces Write

Anonymous ACL submission

Abstract

Misleading variable names can change the values a language model reports even when the correct values remain readable in its internal activity. We extend Kudo et al.’s arithmetic study to Python programs, asking seven models to report each assigned value. Renaming a variable holding a list’s length to suggest a sum makes three models more often write the sum. Yet predictors trained on other programs recover the correct value with 95.5–99.7% accuracy immediately before these wrong writes. Worked examples that include the code expression beside each value sharply reduce the errors; descriptive text of the same length leaves many of them. A correct internal readout therefore cannot certify a written trace: the model can still report the name-suggested value. Including source expressions in worked examples offers a practical way to improve trace reliability.

1 Introduction

A variable named `sum_all` is assigned `len(xs)`. For `[5, 3]`, its value is 2, regardless of its name. Suppose a language model is asked to trace the program: write each assignment’s value, then the return value. In Figure 1, OLMo-2-7B writes 8 for this variable, following the sum suggested by the name. We call this a *name error*.

Changing only the worked examples changes this trace. When examples show values alone, the model writes 8. When they show the code expression beside the value, as in `sum_all = len(xs) = 2`, it writes 2. The program, input and model are unchanged. This contrast motivates our study: how do variable names and the format of worked examples affect which value a model writes?

Earlier work shows that misleading identifiers reduce code accuracy (Le et al., 2025; Lam et al., 2025; Le et al., 2026). We ask whether the correct value remains readable immediately before an in-

correct write: when the model writes 8, can a separate predictor still recover 2? We build on Kudo et al. (2026)’s analysis of intermediate values during arithmetic reasoning. Their probes test what values can be recovered from activations; their patching experiments replace activations to test how outputs change. We apply these tools to code with a deliberate conflict between a variable’s assigned operation and its name.

We train a *linear probe*, a small predictor of the true digit, on separate programs. It reads activations immediately before the model writes a value. We show that this predictor usually returns the computed digit even when the trace follows the misleading name. Including the code expression beside each demonstrated value sharply reduces wrong writes; extra descriptive text of the same length leaves large effects. The worked examples therefore change whether the trace writes the value computed by the code or the value suggested by the name.

2 Study design

Each problem is a short Python function over a list of distinct digits. Assignments compute length, sum, maximum or minimum, or apply integer arithmetic. All correct variable values and return values are single digits. The main level has three dependent assignments (Appendix A). We test seven instruction-tuned models at levels 3 and 5, with 1,000 matched problems per target, with three fixed sets of three worked examples. CodeLlama-34B supplies an additional scale check.

Each pair differs only in one variable name. A congruent name suggests the assigned operation; a misleading name suggests another. The suggested digit must be absent from the inputs, constants and computed values. This excludes maximum and minimum as competing operations on the first as-

Examples with code expressions change the reported values

Task: write each assigned value, then the function's final answer.

Same program and input

```
def f(xs):
    sum_all = len(xs)
    ww = sum_all + 3
    qq = ww + 4
    return qq
f([5, 3])
```

len counts items: 2

sum_all suggests $5 + 3 = 8$

Correct final answer: 9

Examples show values

name = value

Actual OLMo-2-7B output:

sum_all = 8

ww = 11

qq = 15

Answer: 15 (wrong)

Examples include expressions

name = expression = value

Actual OLMo-2-7B output:

sum_all = len(xs) = 2

ww = sum_all + 3
= 5

qq = ww + 4
= 9

Answer: 9 (correct)

Only the worked-example format changes; the model solves the same program.

Figure 1: An observed OLMo-2-7B trace. The name suggests sum 8, but the code counts two items. Including source expressions in worked examples changes the first reported value from 8 to 2 and the final answer from 15 to 9. Only the example format changes.

signment. We also check which operation each model associates with each name. The original main names fail the planned meaning check: agreement in 80% of models. We report that departure and replicate with independently screened names and fresh programs (Appendix L).

Models either give only the return value or first list each variable's value. Worked examples demonstrate values alone ($v = 2$) or expressions with values ($v = \text{len}(xs) = 2$). We also test transcripts resembling an interactive Python session, values added as code comments, and free prose (Appendix D).

Our main measure is *excess name errors*: how often a misleading-name trace writes the suggested digit, minus how often the ordinary-name version writes that same digit. We measure name errors similarly in the direct regime. We call an effect reliable when its sign agrees across all three demonstration sets and a 95% cluster-bootstrap interval excludes zero. To support a small-effect claim, a 90% interval must fit inside the preregistered ± 2 -point margin. Other cases remain inconclusive (Appendix G).

For three models, a linear probe predicts the target digit from a hidden state at one token and layer. Training uses disjoint computations and three probe seeds; layers are selected on neutral programs in other folds before each held-out computation is evaluated. We read the *decision token*: the = immediately before the first traced

value. The generated and cached token prefixes must agree through that token; no value digit is included. Patching replaces the name tokens or this position with a matched neutral twin's activations. A neutral-for-neutral patch measures disruption. Controls and implementation details are in Appendix E.

3 Results

3.1 Listing values can follow a misleading name

Table 1 shows excess sum writes when the code computes a list's length in three affected models. Across ordinary-name programs at this level, their final-answer accuracies are 95.7, 97.4 and 99.6% (full panel: Appendix C).

The sum is not these models' usual error on a length step. On parsed wrong neutral writes, sums account for only 14–32% of errors; maximum values and off-by-one lengths are more common. The sum-like name changes which wrong value they write. CodeLlama-7B's larger effect occurs on a maximum-computed variable (+10.7 points); CodeGemma-7B shows that pairing at level 3. The full pairing matrix is in Appendix C. Direct-answer name-error effects are much smaller: -0.2 to +3.1 points at this level.

Model	Extra sum writes (points)	Correct digit (%) Before trace	Correct digit (%) Before wrong value	Wrong writes (unique programs)
OLMo-2-7B	+58.4	38.4 [32.9, 43.9]	96.4 [94.9, 97.9]	503 (193)
Llama-3.2-3B	+43.4	26.4 [22.4, 30.8]	95.5 [93.6, 97.3]	372 (169)
Llama-3.1-8B	+12.4	56.0 [47.4, 64.2]	99.7 [99.0, 100.0]	106 (45)

Table 1: The code computes length; the name suggests a sum. Extra sum writes are the misleading-name rate minus the ordinary-name rate. Correct-digit predictions are measured on actual wrong sum writes, before the trace starts and before the wrong value. Brackets: 95% intervals.

3.2 The correct digit is decodable on error cases

On actual incorrect sum writes (Figure 2a), neutral-trained probes recover the computed digit with 95.5–99.7% accuracy. Table 1 gives intervals and the number of independent computations; repeated demonstrations remain clustered. The prefixes match through the decision token in all 2,565 generations across the three demonstration sets. The cached state therefore also precedes the model’s actual value write. Appendix K reports clustered intervals and the correct-write and other-error strata.

Five-fold layer selection excludes each test computation. Separately, controls trained to predict a random digit for each name reach 100.0, 99.7 and 87.8% neutral accuracy. These measure name identity; their layers are selected on all neutral programs (Appendix E). Recovering the correct digit on an error establishes decodability, not its use in generation. Correct-digit prediction is less accurate before the trace starts than just before the wrong value is written (Table 1). Both positions use disjoint neutral training and independent layer selection; paired changes are in Appendix M.

Patching independently tests sensitivity to the name and the writing position (Figure 3). Replacing name tokens at layer 0 removes 97% and 100% of name errors in the two models with aligned twins. Name-site removal declines in later layers. At the decision token, removal is weak early and strong from the middle of the network onward; all-layer replacement removes 97–100%. The neutral-word control changes at most 1.0% of outputs. The patches identify positions sensitive to the misleading name. They replace an entire hidden-state vector, so they cannot identify which feature changes the output.

3.3 Including the code expression reduces wrong writes

We keep the programs, models and demonstration sets fixed and change the examples from $v = 2$ to $v = \text{len}(xs) = 2$. Name-error rates fall by 55, 43 and 12 points in the three models (Table 2). OLMo-2-7B retains +3.4 points of excess; the two Llama effects approach zero. Length-matched neutral annotations leave 56.6 and 49.6 points of excess in OLMo and Llama-3.2. Valid numeric elaborations ($v = 2 + 0 = 2$) lower it to 41.3 and 14.9 points, still above source-expression demonstrations (Appendix J). Extra tokens do not reproduce the expression benefit; adding arithmetic accounts for part of it.

Figure 2b shows the format contrast on 200 fresh computations with 3 names selected by meaning checks before observing any traces (Table 16). CodeGemma shows no observed excess in either format. This independent behavioral cohort is separate from the original-name probe cohort; selection and per-name results are in Appendix L.

Interactive Python transcripts also omit expressions, yet their effects are several times smaller; comment traces reduce them further. The larger checkpoint has a smaller maximum-step effect: CodeLlama-34B has +0.0 excess on length steps and +1.8 [+0.7, +3.1] on maximum steps. The latter interval crosses the equivalence margin. Renamed CRUXEval functions show no detectable accuracy effect in direct-answer or prose regimes; those tests do not include the synthetic trace format (Appendix F).

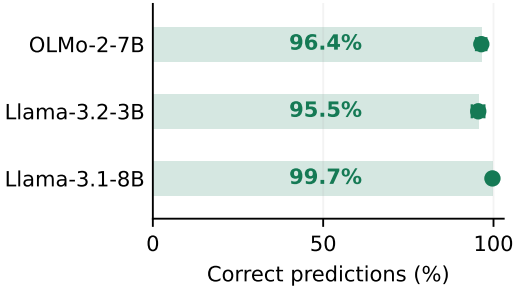
4 Related work

Misleading identifiers affect real-code accuracy (Le et al., 2025; Lam et al., 2025; Le et al., 2026; Orvalho and Kwiatkowska, 2025; Gao et al., 2023). Our synthetic tasks separate computed and name-suggested values. Kudo et al. (2026) study arithmetic readouts; the companion arithmetic study (Anonymous, 2026) tests whether

A wrong write can coexist with a readable correct value

(a) Correct digit read before a wrong sum write

Separate predictor reads internal activity.
Original names; training uses other programs.

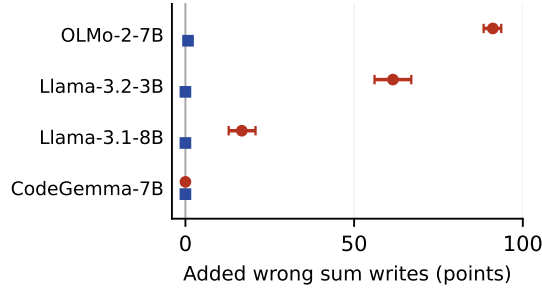


95% intervals. Added writes = misleading-name rate minus ordinary-name rate.

Readable information does not show that the model uses it.

(b) Include the code expression in examples

Fresh programs; names screened before testing.



Examples: $v = 2$
Examples: $v = \text{len}(xs) = 2$

Figure 2: (a) Correct digits predicted immediately before wrong sum writes. (b) Worked examples use $v = 2$ or $v = \text{len}(xs) = 2$; including $\text{len}(xs)$ reduces extra sum writes. The replication uses independently screened names and fresh programs; Table 16 gives intervals.

Format shown in examples	Example	OLMo-2-7B	Llama-3.2-3B	Llama-3.1-8B
Values only	$v = 2$ $>>> v$	+58.4*	+43.4*	+12.4*
Interactive Python transcript	2	+9.5*	+12.6*	+0.2
Code with value comments	$v = \text{len}(xs) \# 2$	+0.0	+6.8*	+0.1
Expression and value	$v = \text{len}(xs) = 2$	+3.4*	+0.4	+0.0

Table 2: Extra sum writes when the code computes length (percentage points). Rows show the output format demonstrated in worked examples; snippets illustrate a value of 2. Larger numbers mean more answers follow the misleading name. *Reliable across demonstration sets; intervals: Table 7.

writing calculations limits misleading-digit answers. Here we test readouts before actual wrong trace writes and change their demonstration format. Faithfulness studies perturb prompts and chains (Turpin et al., 2023; Lanham et al., 2023; Chen et al., 2025); we intervene at the value-writing position. Conflict tasks isolate competing cues (Stroop, 1935; Geirhos et al., 2019); recent language-model studies test lexical responses (Wang, 2026; Hu et al., 2026).

5 Conclusion

Traces listing only values can write a name-suggested value while the correct digit remains decodable. Including the source expression in worked examples reduces these errors, including with independently screened names. Length-matched annotations leave large effects; numeric elaboration helps partly. These results concern the tested names and formats.

Limitations

- **Scope.** Programs are synthetic with single-digit values; the largest effect uses a constructed trace format. Interactive Python transcripts and code comments show smaller effects. The 89-function CRUXEval check bounds direct-answer and prose accuracy, not synthetic trace errors. Models reach 34B; the panel excludes Chinese-origin models, including strong code models in Le et al. (2026).
- **Internal measurements.** Probes on three smaller models establish decodability, not use. Error readouts select layers independently; full-neutral curves and name controls are descriptive. Patches replace mixed representations and may have side effects. Llama-3.1-8B’s `sum_all` lacks an aligned name-site twin.

References

- Anonymous. 2026. Chain of thought reduces errors from misleading names in arithmetic. Under review.
- Yanda Chen et al. 2025. Reasoning models don’t always say what they think. *arXiv preprint arXiv:2505.05410*.
- Shuzheng Gao, Cuiyun Gao, Chaozheng Wang, Jun Sun, David Lo, and Yue Yu. 2023. Two sides of the same coin: Exploiting the impact of identifiers in neural code comprehension. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, pages 1933–1945. ArXiv:2207.11104.
- Robert Geirhos, Patricia Rubisch, Claudio Michaelis, Matthias Bethge, Felix A. Wichmann, and Wieland Brendel. 2019. ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness. In *International Conference on Learning Representations*.
- John Hewitt and Percy Liang. 2019. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*.
- Xiaoyang Hu, Mike Angstadt, Shane Storks, Zan Huang, Aman Taxali, Alex Weigard, Richard L. Lewis, and Chandra Sripada. 2026. Conflict and congruency effects in large language models: In-weight and in-context competition in a verbal conflict task. *arXiv preprint arXiv:2608.11510*.
- Keito Kudo, Yoichi Aoki, Tatsuki Kuribayashi, Shusaku Sone, Masaya Taniguchi, Ana Brassard, Keisuke Sakaguchi, and Kentaro Inui. 2026. LLMs faithfully and iteratively compute answers during CoT: A systematic analysis with multi-step arithmetics. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 1114–1153. Code: <https://github.com/keitokudo/faithful-cot-multistep-arithmetic>.
- Man Ho Lam, Chaozheng Wang, Jen-tse Huang, and Michael R. Lyu. 2025. CodeCrash: Exposing LLM fragility to misleading natural language in code reasoning. In *Advances in Neural Information Processing Systems*, volume 38. ArXiv:2504.14119.
- Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, Kamilë Lukošiušė, Karina Nguyen, Newton Cheng, Nicholas Joseph, Nicholas Schiefer, Oliver Rausch, Robin Larson, Sam McCandlish, Sandipan Kundu, and 11 others. 2023. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*.
- Cuong Chi Le, Minh V. T. Pham, Cuong Duc Van, Hoang N. Phan, Huy N. Phan, and Tien N. Nguyen.

2025. When names disappear: Revealing what LLMs actually understand about code. *arXiv preprint arXiv:2510.03178*.
- Jack Le, Anh H. N. Nguyen, and Tien N. Nguyen. 2026. Do machines struggle where humans do? LLM and human comprehension of obfuscated code. *arXiv preprint arXiv:2606.31725*.
- Pedro Orvalho and Marta Kwiatkowska. 2025. Are large language models robust in understanding code against semantics-preserving mutations? *arXiv preprint arXiv:2505.10443*.
- J. Ridley Stroop. 1935. Studies of interference in serial verbal reactions. *Journal of Experimental Psychology*, 18(6):643–662.
- Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. 2023. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, volume 36, pages 74952–74965.
- Han-yu Wang. 2026. Persistent priors, preserved targets: A Stroop-style paradigm for lexical override. *arXiv preprint arXiv:2606.07555*.
- Fred Zhang and Neel Nanda. 2024. Towards best practices of activation patching in language models: Metrics and methods. In *International Conference on Learning Representations*.

A Task levels and identifiers

Levels vary dependency depth, distractors and which assignment receives the misleading name. The primary comparison uses level 5. A function is generated once; renaming preserves its statements, input and executable result. Table 4 lists the operation suggested by each identifier. Sum and length are admissible name-suggested operations on the aggregate step; minimum and maximum are excluded because their result is already present in the input.

B Identifier meanings

For each identifier and model, we compare log-probabilities of candidate right-hand sides after a prefix such as `def f(xs):\n` `total =`. The highest-scoring operation records what that identifier evokes. Of 25 names, 12 match the intended operation in at least 80% of models. Most sum-family names instead evoke length in 2 of seven models. This check matters when interpreting a small effect: an identifier need not suggest the same operation to every model. The design planned to retain only names passing the 80%

Level	Neutral program (body)	Traced steps
1	<code>v = len(xs); return v</code>	1
2	<code>v = len(xs); w = v * 2; return w</code>	2
3	<code>v = len(xs); w = max(xs); u = v + w; return u</code>	3
4	level 3 with an unused fourth assignment	3
5	<code>v = len(xs); w = v * 2; u = w + 3; return u</code>	3
6	level 5 with the misleading name on the middle step	3

Table 3: The levels, one example each. The aggregate in the first statement is any of len, sum, max, min; unary operations are `*2`, `//2`, `+c`, `-c`. The misleading name sits on the first statement at levels 1–5 and on the second at level 6.

Family	Names	Implied operation
len	count, n_items, length	<code>len(xs)</code>
sum	total, sum_all, acc	<code>sum(xs)</code>
max	largest, max_val, peak	<code>max(xs)</code>
min	smallest, min_val, low	<code>min(xs)</code>
double	double, twice	<code>2 * a</code>
half	half	<code>a // 2</code>
succ	succ, next_val	<code>a + 1</code>
pred	pred, prev_val	<code>a - 1</code>
add	combined, both	<code>a + b</code>
sub	diff, gap	<code>a - b</code>
mul	product, scaled	<code>a * b</code>

Table 4: Identifier families and their suggested operations. The gate (Appendix B) scores each name by the log-probability each model assigns to the candidate right-hand sides; 12 of 25 names are read as intended by at least 80% of models. The min and max names cannot produce name errors on the aggregate step (their value is an input element) and are used as congruent and alternative-name names.

panel gate. The runs instead retained the full name table, so the reported comparisons depart from that selection rule. The results concern these named groups; they do not establish that every model reads each name as intended.

C Operation and name pairings

Table 6 reports all tested aggregate/name pairings. The reverse of the main length-to-sum conflict is weak at level 5: sum-computed variables with length-like names have excess within two points for models above the neutral-accuracy floor. At level 3, OLMo-2-7B has +4.4 points of excess in that pairing. Gemma-3-4B shows no main-cell effect, consistent with its identifier-meaning check. OLMo-2-1B’s neutral accuracy is only 37.8%, making its small, broadly distributed effects harder to interpret as a specific trace failure.

D Demonstration formats

All few-shot comparisons hold programs, demonstration identities and model weights fixed. The values-only format writes `qq = 3`; the expression-and-value format writes `qq = len(xs) = 3`; the interactive Python format writes `>>> qq` followed by

3; comment mode annotates each assignment with its value. These are model-generated transcripts, not interpreter executions. Prose asks for step-by-step reasoning without demonstrations. Table 7 gives the paired excess and interval for each few-shot format.

On the main conflict, interactive-transcript excess is +9.5 and +12.6 points for OLMo-2-7B and Llama-3.2-3B, compared with +0.2 for Llama-3.1-8B. Comments leave +6.8 points in Llama-3.2-3B and near-zero excess in the other two. Prose has little initial name errors, so asking it to state or recompute values does not test removal of the large trace effect. The prediction that interactive transcripts would stay within a factor of two of trace was not supported; the comment prediction held only for two models.

The paired trace-minus-trace-expr name-error-rate reductions are 55 points [49.9, 60.2], 43 [38.2, 47.7] and 12 [8.8, 16.0] for OLMo-2-7B, Llama-3.2-3B and Llama-3.1-8B. These 95% intervals keep each matched program’s demonstration-seed repeats together.

Model	Return acc.	Direct: errors	Trace: len→sum	Trace: max→sum
OLMo-2-7B	95.7	+0.4	+58.4* [+53.0, +63.5]	+1.9 [+1.1, +3.0]
Llama-3.2-3B	97.4	+0.8	+43.4* [+38.5, +48.1]	+0.0 [+0.0, +0.0]
Llama-3.1-8B	99.6	+0.5	+12.4* [+8.8, +16.0]	+0.0 [+0.0, +0.0]
CodeLlama-7B	91.7	+1.0	+3.7* [+2.2, +5.4]	+10.7* [+7.9, +13.7]
CodeGemma-7B	99.2	+1.5*	+0.1 [+0.0, +0.4]	+0.0 [+0.0, +0.0]
Gemma-3-4B	99.1	+0.8	+0.0 [+0.0, +0.0]	+0.0 [+0.0, +0.0]
OLMo-2-1B	37.8	+3.1*	+3.7* [+2.2, +5.5]	+4.2* [+2.6, +5.9]
CodeLlama-34B	98.9	–	+0.0 [+0.0, +0.0]	+1.8* [+0.7, +3.1]

Table 5: Full level-5 panel: ordinary-name final-answer accuracy (%) and excess name errors (points). Direct contrasts rename the returned variable; trace contrasts rename the first assignment. Trace columns include 95% intervals. Stars mark consistent signs and intervals excluding zero.

Model	len→sum	max→sum	min→sum	sum→len	max→len
OLMo-2-7B	+17.6* / +58.4*	+0.9 / +1.9	+0.0 / +0.2	+4.4* / +0.0	+0.0 / +0.0
Llama-3.2-3B	+23.6* / +43.4*	+1.1 / +0.0	+0.0 / +0.0	+0.0 / +0.0	+0.0 / +0.0
Llama-3.1-8B	+1.5 / +12.4*	+0.7 / +0.0	+0.0 / +0.0	+0.0 / +0.0	+0.0 / +0.0
CodeLlama-7B	+0.9 / +3.7*	+7.5 / +10.7*	+1.0 / +1.7	+1.5 / +0.0	+0.9 / +0.6
CodeGemma-7B	+0.0 / +0.1	+10.8* / +0.0	+0.0 / +0.0	+0.0 / +0.0	+0.0 / +0.0
Gemma-3-4B	+0.0 / +0.0	+0.2 / +0.0	+0.0 / +0.0	+0.0 / +0.0	+0.0 / +0.0
OLMo-2-1B	+1.2 / +3.7*	+2.8* / +4.2*	+0.3 / -0.2	+10.6* / +11.1*	+9.6* / +5.6*

Table 6: Excess name errors at the aggregate step, trace regime, level 3 / level 5, three sets of worked examples pooled; * as in Table 5.

E Probe and patching details

We follow Kudo et al.’s 10-class linear-probe recipe, with three seeds and neutral-only training. Each probe predicts a target digit from one residual-stream position and layer. Name-identity controls use deterministic random labels keyed on the identifier (Hewitt and Liang, 2019). Neutral names and values vary independently, and misleading names are absent from training. The original implementation reused test programs in training. The reported reruns replace those scores: they train on 2,000 independent neutral programs and evaluate 285 misleading-name programs, with overlap guards that ignore name spelling. Table 8 reports the name controls alongside digit accuracy.

Figure 3’s probe curves use demonstration set 7. Repeating training and evaluation with sets 11 and 13 gives true-digit accuracies of 95–97%, 94–100% and 100–100% across the three sets. Each repeat uses the same independent training programs and three probe seeds; its layer is selected by that set’s neutral accuracy. These repeats assess prompt sensitivity, not a causal role for the decoded digit. Table 8 gives both conditions’ readouts and controls.

We supply the correct trace token by token when caching probe inputs. The decision position is the = before the first traced value; later

values cannot affect this position under causal attention. Patching uses only the prompt and this trace prefix. Name patches replace every aligned occurrence of the identifier; decision patches replace only that writing position. Both use one layer at a time and include a neutral-for-neutral disruption control (Zhang and Nanda, 2024). The alternative-name control is uninformative in the main cell: every sum-family identifier implies the same sum, so there is no distinct source name-suggested value. Llama-3.1-8B’s affected two-token name lacks aligned neutral twins for the name-site intervention.

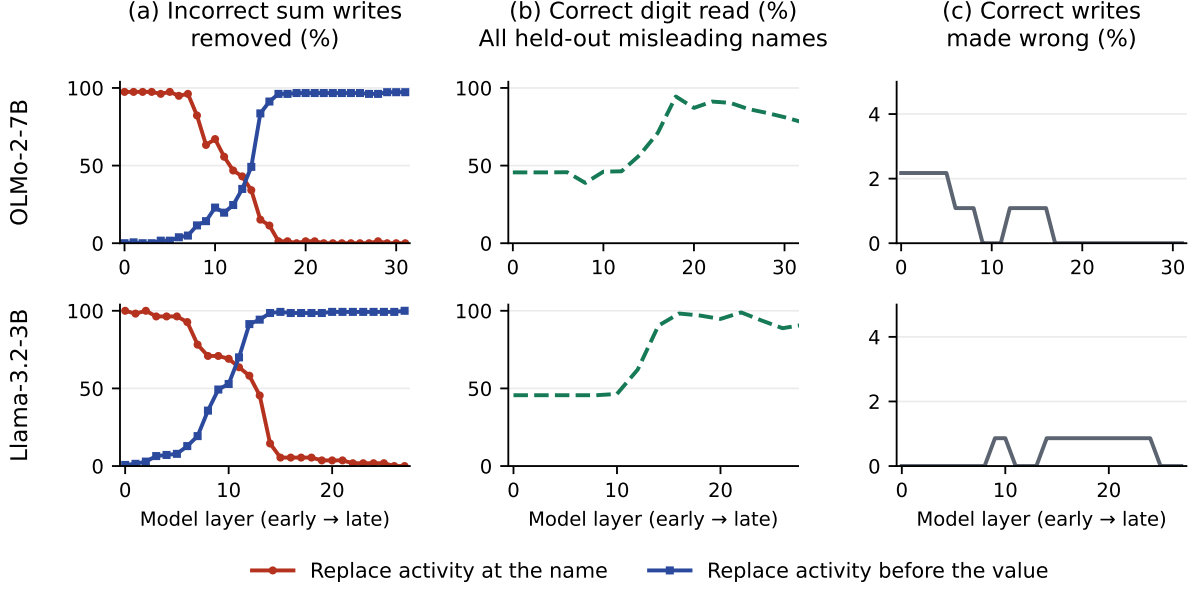
F Scope checks

Operation and scale. Moving the misleading name to a unary middle step produces no reliable effect for models above the accuracy floor; the largest excess is +0.6 points. CodeLlama-34B’s length-to-sum effect is +0.0, while its maximum-to-sum effect is +1.8 points [+0.7, +3.1]. The latter is a reliable positive effect, but its interval extends beyond the two-point margin. Its small point estimate cannot be reported as equivalence. The larger-checkpoint result supports the registered maximum-to-sum direction and reliable-effect criterion, while its magnitude is much smaller than CodeLlama-7B’s.

Format shown in examples	Example	OLMo-2-7B	Llama-3.2-3B	Llama-3.1-8B
Values only	<code>v = 2</code> <code>>>> v</code>	+58.4* [+53.0, +63.5]	+43.4* [+38.5, +48.1]	+12.4* [+8.8, +16.0]
Interactive Python transcript	<code>2</code>	+9.5* [+6.9, +12.0]	+12.6* [+9.4, +15.9]	+0.2 [+0.0, +0.6]
Code with value comments	<code>v = len(xs) # 2</code>	+0.0 [+0.0, +0.0]	+6.8* [+4.4, +9.2]	+0.1 [+0.0, +0.4]
Expression and value	<code>v = len(xs) = 2</code>	+3.4* [+2.0, +5.1]	+0.4 [+0.0, +0.9]	+0.0 [+0.0, +0.0]

Table 7: Table 2 with 95% bootstrap intervals over matched problems.

Replacing internal activity can change wrong sum writes



Replacement uses the ordinary-name twin; removing a sum write can produce another wrong value.

Figure 3: **(a)** Incorrect sum writes removed by replacing internal activity with the ordinary-name program’s activity, at the name (red) or just before the value (blue). **(b)** A separate predictor reads the correct digit on held-out misleading-name programs. **(c)** Correct writes made wrong by name replacement; note the smaller scale. Columns show distinct measurements, not a common error rate.

Natural code. We select 89 CRUXEval functions with length, loop-counter or maximum assignments, rename the target by AST rewriting to `v` or `total`, and verify execution after renaming. Direct and prose accuracy contrasts all have intervals covering zero on the 84 functions whose renamed target is used downstream. Their endpoints stay within 10 and 13 points, respectively (Table 10). These are bounded results for those functions and prompts; they do not test the trace that lists values without expressions used in the synthetic study.

G Effect and equivalence checks

Each contrast subtracts the neutral twin’s rate for the same digit. We resample matched problems with demonstration-seed replicates together and use 95% cluster-bootstrap intervals. Reliability

also requires a consistent sign across all three demonstration sets.

Equivalence applies the ± 2 -point margin to 90% intervals from the same cluster bootstrap (two one-sided 5% tests), using 2,000 draws. Across 192 tested contrasts, 150 intervals lie wholly inside the margin, 36 contrasts have reliable effects without meeting that bound, and 6 meet neither criterion. A small estimate or an interval covering zero does not by itself establish equivalence.

H Generated-prefix audit and initial readouts

For 2565 of 2565 misleading-name generations, the token prefix through the marker before the first value exactly matches the cached trace. This verifies that the cached state at that marker also

Model	Set	Layer	Neutral	Control	Misleading	Control	Majority
OLMo-2-7B	7	18	99.5	100.0	94.5	33.0	45.6
OLMo-2-7B	11	18	97.3	100.0	95.4	0.0	45.6
OLMo-2-7B	13	18	99.8	100.0	97.4	23.4	45.6
Llama-3.2-3B	7	24	98.8	99.7	93.7	20.0	45.6
Llama-3.2-3B	11	20	100.0	99.8	95.4	5.0	45.6
Llama-3.2-3B	13	28	99.4	100.0	99.9	1.9	45.6
Llama-3.1-8B	7	16	100.0	87.8	99.9	33.0	45.6
Llama-3.1-8B	11	20	100.0	81.6	100.0	0.1	45.6
Llama-3.1-8B	13	20	100.0	87.2	100.0	3.0	45.6

Table 8: Disjoint code probes, three demonstration sets. Neutral and misleading are true-digit accuracy; each control column gives name-identity accuracy for that condition. Majority predicts the most frequent digit in training on the misleading test programs. Rates are percentages, averaged over three probe seeds. Layers are selected by neutral accuracy.

Model	Name tokens			Decision token			
	L0	damage	<10% by	50% at	90% at	all layers	damage
OLMo-2-7B	97%	2%	L17	L15	L16	97%	2%
Llama-3.2-3B	100%	0%	L15	L10	L12	100%	0%
Llama-3.1-8B	–	–	–	L8	L9	100%	0%

Table 9: Layers behind Figure 3. Name tokens: name errors removed by replacing every token of the name at layer 0, damage to correct instances, and the first layer after which removal falls below 10%. Decision token: first layer at which a single-layer replacement removes 50% and 90% of name errors, removal with every layer replaced, and its damage. Llama-3.1-8B’s name errors fall on a two-token name without a token-aligned twin.

precedes the generated value. No target value token is included. These initial readouts select layers on the full neutral evaluation set; the main text instead uses independent cross-fitted selection (Appendix K). We split the generated writes into name-suggested values, correct values and other errors. Table 11 averages three probe seeds and pools demonstration sets 7, 11 and 13. Intervals resample matched programs with all their demonstration repeats (4,000 draws). These conditional readouts establish available information on observed errors; they do not establish its causal use.

I Sensitivity to identifier meanings

None of the three main-cell names (total, sum_all, acc) meets the originally planned 80% panel gate. Its filtered estimate is therefore unavailable, rather than zero. Table 12 retains the full-data estimate and reports each name and each model’s own meaning gate. Gates use the original saved meaning checks. Intervals cluster matched programs across the three demonstration sets. This exploratory sensitivity does not replace the planned panel gate or remove the deviation from it.

J Length and numeric elaboration controls

We add neutral annotations to each demonstrated value, matching the expression demonstrations’ token counts, and separately add valid numeric elaborations of the form name = value + 0 = value. The test program and Trace: prefix are unchanged. Greedy decoding uses the same 285 misleading programs and neutral twins for each format. Table 13 pools three demonstration sets; changes relative to the original trace and expression trace are paired on the same programs. The length-matched annotation controls retain much larger excess than expression demonstrations in both models. Numeric elaboration reduces excess relative to the original trace but leaves a larger effect than source expressions. Token count alone does not reproduce the expression benefit, and adding valid arithmetic accounts for part of it; these two contrasts do not isolate a single mechanism.

K Independent probe layer selection

The main-text error readouts use five-fold cross-fitting. A fixed SHA256 hash assigns canonical program computations to folds, keeping renamed versions and demonstration repeats together. For

Model	Direct acc.	Direct Δ	Prose acc.	Prose Δ
OLMo-2-7B	10	+1.2 [+0.0, +3.6]	22	-1.2 [-9.5, +8.3]
Llama-3.2-3B	19	+1.2 [-2.4, +4.8]	25	+3.6 [-3.6, +10.7]
Llama-3.1-8B	22	-3.6 [-9.5, +2.4]	40	+0.0 [-9.5, +9.5]
CodeLlama-7B	18	+2.4 [+0.0, +6.0]	13	+2.4 [-4.8, +9.5]
CodeGemma-7B	18	+0.0 [-4.8, +4.8]	22	+2.4 [-8.3, +13.1]
Gemma-3-4B	15	-2.4 [-6.0, +0.0]	45	+4.8 [-1.2, +11.9]
OLMo-2-1B	16	-3.6 [-8.3, +0.0]	3	+6.0 [+0.0, +11.9]

Table 10: 89 CRUXEval functions that assign a variable from `len(. . .)` (48), a loop counter (38) or `max(. . .)` (3), of which 84 use it downstream; each rendered with the original name, the neutral name `v`, and the misleading name `total`. Accuracy on all neutral renderings, and the paired misleading-minus-neutral difference in points on the downstream-used subset, with a bootstrap interval over functions.

Model	Generated write	Rows	Programs	True-digit accuracy (%)
OLMo-2-7B	name error	503	193	96.4 [94.8, 97.8]
OLMo-2-7B	correct write	320	135	96.7 [94.4, 98.5]
OLMo-2-7B	other error	32	24	77.1 [61.7, 90.5]
Llama-3.2-3B	name error	372	169	95.7 [93.8, 97.4]
Llama-3.2-3B	correct write	440	204	98.3 [97.0, 99.3]
Llama-3.2-3B	other error	43	29	82.2 [71.2, 91.7]
Llama-3.1-8B	name error	106	45	99.7 [99.0, 100.0]
Llama-3.1-8B	correct write	749	261	100.0 [100.0, 100.0]
Llama-3.1-8B	other error	0	0	–

Table 11: Disjoint probes at the verified generated prefix. A dash denotes an empty stratum. The denominator is observed writes within each stratum, not independent demonstration repeats.

each held-out fold, we select the layer with the highest neutral accuracy on the other four folds, averaging three probe seeds and breaking ties toward the lowest layer. Neither the held-out neutral labels nor its misleading writes enter that choice. Probe training remains disjoint from every evaluation computation. Each program is scored only with its held-out fold’s layer. The generated-prefix audit from Appendix H still applies. Table 14 reports the conditional write strata and paired changes from the earlier full-neutral selection. Intervals use 4,000 canonical-program bootstrap draws, keeping demonstration repeats together. They condition on the fixed fold selectors and do not refit layer selection inside each bootstrap. Name controls and the full layer curves remain descriptive measurements in Appendix E.

L Independent replication with screened names

We fix 16 fresh sum-family candidates before scoring them with the original candidate-continuation meaning check (Appendix B). The panel is the same seven models used for that gate. A name qualifies when sum is the highest-scoring continuation in at least 80% of the panel; ties follow the original scoring order. Two additional

function-name contexts and neutral names are diagnostic and do not affect selection. Of the candidates, 14 qualify. We freeze the first 3 qualifying names in the fixed order: `sum_of_values`, `sum_of_items`, `sum_of_elements`. No trace outcomes enter selection. The replication uses 200 new length-computed programs. Their canonical computations are unique and absent from the original level-5 training and test corpus and all three demonstration sets. True and name-suggested values are checked by execution and retain the original value-exclusion rules. Every selected name is crossed with every program; its neutral twin differs only in the target identifier. The three affected models and the prespecified CodeGemma-7B control use the same programs and demonstration sets 7, 11 and 13 under trace and expression demonstrations. Table 16 pools names and demonstration sets, clustering all repeats of each computation in 4,000 bootstrap draws. Table 17 reports each identifier. The format contrast replicates on independently selected names, but names and programs differ from the original experiment, so differences in effect magnitude across the two samples are descriptive. This exploratory replication does not retroactively repair the original selection-rule deviation. Probes and patches concern the original

Model	All	Model gate	total	sum_all	acc
Gemma-3-4B	0.0 [0.0, 0.0]	–	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]
CodeGemma-7B	0.1 [0.0, 0.4]	0.1 [0.0, 0.4]	0.0 [0.0, 0.0]	0.3 [0.0, 1.0]	0.0 [0.0, 0.0]
Llama-3.1-8B	12.4 [8.9, 16.0]	12.4 [8.9, 16.0]	0.0 [0.0, 0.0]	34.0 [26.0, 42.3]	0.0 [0.0, 0.0]
OLMo-2-1B	3.7 [2.2, 5.5]	5.0 [1.8, 8.9]	5.0 [1.8, 8.9]	5.8 [2.6, 9.3]	0.0 [0.0, 0.0]
OLMo-2-7B	58.4 [53.1, 63.4]	59.3 [52.7, 65.8]	56.4 [47.9, 64.9]	96.2 [93.9, 98.1]	15.3 [9.6, 21.8]
CodeLlama-7B	3.7 [2.1, 5.5]	3.7 [2.1, 5.5]	1.8 [0.4, 3.2]	8.7 [4.5, 13.1]	0.0 [0.0, 0.0]
Llama-3.2-3B	43.4 [38.5, 48.0]	43.4 [38.5, 48.0]	30.1 [23.8, 36.9]	76.3 [70.2, 82.4]	18.4 [11.9, 25.7]

Table 12: Excess name errors in percentage points [95% interval], level 5 length-to-sum. Empty subsets are shown as dashes. The saved JSON also reports each demonstration set and the all-three-sets sign rule.

Model	Format	Excess	Change vs trace	Change vs expression
OLMo-2-7B	neutral annotation	56.6 [51.3, 61.6]	-1.8 [-3.4, -0.1]	53.2 [48.1, 58.2]
OLMo-2-7B	numeric elaboration	41.3 [36.4, 46.0]	-17.1 [-21.1, -12.9]	37.9 [33.2, 42.5]
Llama-3.2-3B	neutral annotation	49.6 [44.7, 54.3]	6.2 [4.1, 8.4]	49.2 [44.3, 53.9]
Llama-3.2-3B	numeric elaboration	14.9 [12.0, 17.8]	-28.5 [-32.7, -24.3]	14.5 [11.7, 17.4]

Table 13: Excess name errors and paired changes in percentage points [95% interval]. Bootstrap clusters keep all demonstration repeats for each matched program together. These controls assess length and numeric elaboration; they do not by themselves establish how expressions change generation.

identifiers, not these new names.

M Prompt-end versus pre-write readouts

We fit separate probes at the final token of the complete prompt ending in `Trace:`, before generating any trace token. Tokenization of this prefix is checked against the original continuation. Training computations, demonstration sets, SGD recipe and seeds match the pre-write probes. Each position selects its layer independently using the same five canonical-computation folds and other folds’ neutral labels. Outcome strata and generated-prefix eligibility are frozen from the previous audit. Intervals use 4,000 paired canonical-computation bootstrap draws and condition on the fixed selectors.

Model	Write	Rows	Computations	Accuracy (%)	Change (points)
OLMo-2-7B	name error	503	193	96.4 [94.9, 97.9]	0.0 [0.0, 0.0]
OLMo-2-7B	correct write	320	135	96.7 [94.4, 98.4]	0.0 [0.0, 0.0]
OLMo-2-7B	other error	32	24	77.1 [61.6, 90.6]	0.0 [0.0, 0.0]
Llama-3.2-3B	name error	372	169	95.5 [93.6, 97.3]	-0.2 [-0.9, 0.4]
Llama-3.2-3B	correct write	440	204	98.0 [96.6, 99.1]	-0.3 [-0.8, 0.0]
Llama-3.2-3B	other error	43	29	82.2 [71.1, 91.7]	0.0 [0.0, 0.0]
Llama-3.1-8B	name error	106	45	99.7 [99.0, 100.0]	0.0 [0.0, 0.0]
Llama-3.1-8B	correct write	749	261	100.0 [99.9, 100.0]	-0.0 [-0.1, 0.0]
Llama-3.1-8B	other error	0	0	–	–

Table 14: True-digit readouts with independent layer selection. Changes compare the same observed writes with the earlier full-neutral layer choice. A dash denotes an empty stratum. These readouts establish decodability, not causal use.

Candidate	Sum agreement	Qualifies	Selected
sum_of_values	6/7	yes	yes
sum_of_items	7/7	yes	yes
sum_of_elements	7/7	yes	yes
sum_of_xs	7/7	yes	no
sum_of_list	6/7	yes	no
list_sum	6/7	yes	no
input_sum	6/7	yes	no
items_sum	6/7	yes	no
elements_sum	5/7	no	no
sum_values	5/7	no	no
sum_result	6/7	yes	no
sum_total	6/7	yes	no
total_sum	6/7	yes	no
xs_sum	7/7	yes	no
sum_xs	7/7	yes	no
summation	6/7	yes	no

Table 15: The entire fixed candidate list, in selection order. Qualification uses the original canonical meaning prompt. Selection retains the first three qualifying candidates without consulting behavior.

Model	Trace excess	Expression excess	Expression minus trace
OLMo-2-7B	91.1 [88.4, 93.6]	0.8 [0.3, 1.3]	-90.3 [-92.9, -87.6]
Llama-3.2-3B	61.5 [56.1, 66.9]	0.0 [0.0, 0.0]	-61.5 [-66.9, -56.1]
Llama-3.1-8B	16.7 [12.8, 20.8]	0.0 [0.0, 0.0]	-16.7 [-20.8, -12.8]
CodeGemma-7B	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]

Table 16: Independent screened-name replication: excess name errors and paired format changes in percentage points [95% interval]. All selected identifiers and three demonstration sets are pooled over 200 fresh computations. A zero bootstrap interval records no observed paired name errors; it is not a population-level absence or equivalence claim.

Model	Identifier	Trace excess	Expression excess
OLMo-2-7B	sum_of_values	91.7 [88.8, 94.3]	0.2 [0.0, 0.5]
OLMo-2-7B	sum_of_items	89.2 [86.0, 92.0]	2.0 [0.8, 3.3]
OLMo-2-7B	sum_of_elements	92.5 [89.8, 94.8]	0.2 [0.0, 0.5]
Llama-3.2-3B	sum_of_values	61.7 [56.0, 67.3]	0.0 [0.0, 0.0]
Llama-3.2-3B	sum_of_items	55.5 [50.0, 61.3]	0.0 [0.0, 0.0]
Llama-3.2-3B	sum_of_elements	67.3 [61.7, 73.0]	0.0 [0.0, 0.0]
Llama-3.1-8B	sum_of_values	12.8 [9.3, 16.7]	0.0 [0.0, 0.0]
Llama-3.1-8B	sum_of_items	22.8 [18.0, 27.7]	0.0 [0.0, 0.0]
Llama-3.1-8B	sum_of_elements	14.5 [10.5, 18.7]	0.0 [0.0, 0.0]
CodeGemma-7B	sum_of_values	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]
CodeGemma-7B	sum_of_items	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]
CodeGemma-7B	sum_of_elements	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]

Table 17: Per-identifier replication, pooling the same three demonstration sets. Effects are percentage points [95% canonical-program bootstrap interval]. All affected-model trace means are positive in every demonstration set; per-set estimates are retained in the saved summary.

Model	Write	Rows / computations	Prompt	Before write	Change	Name ID
OLMo-2-7B	sum write	503 / 193	38.4 [32.9, 43.9]	96.4 [94.9, 97.9]	+58.1 [+52.6, +63.4]	12.7 [9.8, 15.9]
OLMo-2-7B	correct write	320 / 135	35.2 [27.7, 43.1]	96.7 [94.4, 98.4]	+61.5 [+53.3, +69.3]	20.8 [15.5, 26.3]
OLMo-2-7B	other error	32 / 24	76.0 [58.1, 90.2]	77.1 [61.6, 90.6]	+1.0 [-19.6, +22.6]	26.0 [12.6, 41.1]
Llama-3.2-3B	sum write	372 / 169	26.4 [22.4, 30.8]	95.5 [93.6, 97.3]	+69.1 [+64.5, +73.4]	0.7 [0.1, 1.6]
Llama-3.2-3B	correct write	440 / 204	37.9 [32.1, 43.7]	98.0 [96.6, 99.1]	+60.1 [+54.4, +65.8]	3.6 [1.8, 5.7]
Llama-3.2-3B	other error	43 / 29	20.9 [11.7, 31.7]	82.2 [71.1, 91.7]	+61.2 [+46.8, +73.8]	2.3 [0.0, 6.1]
Llama-3.1-8B	sum write	106 / 45	56.0 [47.4, 64.2]	99.7 [99.0, 100.0]	+43.7 [+35.4, +52.4]	0.0 [0.0, 0.0]
Llama-3.1-8B	correct write	749 / 261	49.9 [45.7, 54.1]	100.0 [99.9, 100.0]	+50.1 [+45.8, +54.3]	0.0 [0.0, 0.0]
Llama-3.1-8B	other error	0 / 0	—	—	—	—

Table 18: Paired true-digit accuracies and prompt-position name-identity controls (%) [95% interval]. Change is pre-write minus prompt accuracy in points.